

Bàn về tư duy “phân” và “hợp” trong tin học

Yang Mu

Trường Trung học số 3 Phúc Châu, Phúc Kiến

2007

Nguồn gốc: An elementary analysis of “division” and “combination” in informatics

IOI China candidate team papers / 2007 / day 1 / Yang Mu

Tóm tắt

Bài viết thảo luận việc vận dụng tư duy “phân” và “hợp” trong tin học. Cốt lõi của tư duy này là tìm ra bản chất của bài toán qua việc chuyển đổi qua lại giữa tách nhỏ và gom lại, từ đó giải quyết bài toán. Phạm vi của nó không chỉ giới hạn trong “chia để trị” theo nghĩa truyền thống. Thông qua ba ví dụ *Mẫu sữa*, *Tái dựng cây* và *Dãy tối ưu*, bài viết giới thiệu cách dùng cụ thể của tư duy “phân” và “hợp” trong quá trình giải bài.

Từ khóa. Tư duy phân và hợp; chuyển hóa; quan hệ biện chứng.

1 Dẫn nhập

“Thế lớn trong thiên hạ, hợp lâu tất phân, phân lâu tất hợp” là câu mở đầu giàu ý vị biện chứng và có thể xem là tinh thần của *Tam quốc diễn nghĩa*. Lịch sử lâu dài cũng nhiều lần cho thấy sự chuyển hóa giữa hợp và phân: một quá trình phát triển xoáy ốc, từ thấp đến cao, trong đó hai mặt ấy liên tục tác động lẫn nhau.

Quy luật này không chỉ xuất hiện trong đời sống xã hội. Trong nhiều bài toán tin học, cách đi từ trạng thái “phân” sang “hợp”, hoặc ngược lại, cũng bộc lộ rất rõ trong quá trình tìm lời giải.

Tư duy “phân” là chia một bài toán lớn, khó xử lý trực tiếp, thành những bài toán con nhỏ hơn hoặc thành các trường hợp có thêm ràng buộc, rồi từ đó tìm đường giải. Tư duy “hợp” đi theo chiều ngược lại: gom các mảnh nhỏ đã tách ra thành một bài toán lớn hơn, qua đó làm rõ quan hệ giữa chúng và rút ra phương pháp.

Trong các kỳ thi tin học, ta thường gặp bài toán có quy mô quá lớn nên khó quan sát, hoặc có quá nhiều tình huống khiến việc suy nghĩ trở nên nặng nề. Khi ấy, thử nhìn bài toán bằng cặp tư duy “phân” và “hợp” thường giúp tìm được điểm vào.

2 Phân tích ví dụ

Sau đây là ba ví dụ: *Mẫu sữa*, *Tái dựng cây* và *Dãy tối ưu*. Mỗi ví dụ dùng một khía cạnh khác nhau của tư duy “phân” và “hợp”.

2.1 Ví dụ 1: Mẫu sữa

Phát biểu. Do chất lượng sữa của đàn bò dao động từng ngày, Farmer John muốn tìm quy luật dao động ấy. Ông ghi lại một dãy

$$a_1, a_2, \dots, a_N$$

gồm các số nguyên từ 0 đến 1 000 000, với $1 \leq N \leq 20\,000$. Cần tìm độ dài lớn nhất của một đoạn mẫu xuất hiện trong dãy ít nhất k lần. Ví dụ, trong dãy 1, 2, 3, 2, 3, 2, 3, 1, đoạn 2, 3, 2, 3 xuất hiện hai lần, nên độ dài mẫu lớn nhất là 4.

Bài này lấy từ USACO December 2006 Gold, *Milk Patterns*.

Phân tích ban đầu. Đây là bài toán liên quan đến so khớp chuỗi. Một công cụ tự nhiên là cây hậu tố; thật vậy, cây hậu tố giải được bài toán trong thời gian tuyến tính. Tuy nhiên độ phức tạp cài đặt cao khiến nó không phải lựa chọn thuận tiện trong thời gian thi. Ta cần tìm một cách đơn giản hơn mà vẫn đủ hiệu quả.

Liệt kê trực tiếp. Cách đơn giản nhất là xét mọi đoạn con của dãy, so sánh chúng với nhau, đếm số lần xuất hiện của từng đoạn rồi lấy đoạn dài nhất xuất hiện ít nhất k lần. Số đoạn con đạt cỡ N^2 , còn việc so sánh hai đoạn có thể tốn $\mathcal{O}(N)$, vì vậy cách này có độ phức tạp đến $\mathcal{O}(N^5)$ nếu làm thô theo từng cặp và từng vị trí. Nó hoàn toàn không đáp ứng được giới hạn.

Nguyên nhân chính của sự kém hiệu quả là có quá nhiều phép so sánh lặp lại. Ta có thể nghĩ đến KMP: sau khi chọn một đoạn con làm mẫu, dùng KMP để đếm số lần nó xuất hiện trong dãy trong $\mathcal{O}(N)$ thời gian. Khi đó tổng độ phức tạp còn $\mathcal{O}(N^3)$. Dù vậy, việc tiếp tục cải thiện bằng các thuật toán so khớp chuỗi trực tiếp gặp nhiều trở ngại. Ta quay lại điểm xuất phát và đổi góc nhìn.

Cải tiến cách xét. Quan sát cách làm thô: việc xét mọi đoạn con có nhiều phần dư thừa, bởi hai đoạn chỉ có thể bằng nhau nếu chúng cùng độ dài. Gợi ý này dẫn đến việc xét theo độ dài đoạn.

Bài toán có một tính đơn điệu rõ ràng. Nếu tồn tại đoạn con S_{len} độ dài len xuất hiện ít nhất k lần, thì với mọi $\text{len}' < \text{len}$, cũng tồn tại một đoạn con $S_{\text{len}'}$ độ dài len' xuất hiện ít nhất k lần: chỉ cần lấy cùng một vị trí tương đối trong mỗi bản sao của S_{len} . Vì vậy ta có thể tìm kiếm nhị phân đáp án. Với một độ dài cố định len , chỉ cần kiểm tra xem có đoạn độ dài len nào xuất hiện ít nhất k lần hay không.

Sau khi đổi cách xét, nút thắt mới là kiểm tra hai đoạn cùng độ dài có bằng nhau không. Với độ dài len , chỉ có $N - \text{len} + 1$ đoạn ứng viên. Ta lưu chúng trong bảng băm và dùng giá trị băm để nhận diện.

Với đoạn

$$\text{sub}_t = (a_t, a_{t+1}, \dots, a_{t+\text{len}-1}),$$

chọn hai số nguyên tố p_1, p_2 và hai mô-đun lớn $\text{hashsize}_1, \text{hashsize}_2$. Đặt

$$f_t = \left(\sum_{j=0}^{\text{len}-1} a_{t+j} p_1^{\text{len}-1-j} \right) \bmod \text{hashsize}_1,$$
$$\text{pos}_t = \left(\sum_{j=0}^{\text{len}-1} a_{t+j} p_2^{\text{len}-1-j} \right) \bmod \text{hashsize}_2.$$

Giá trị pos_t cho biết vị trí trong bảng băm, còn f_t thay cho nội dung đoạn khi cần xử lý va chạm. Nếu hai đoạn có cả hai giá trị pos và f giống nhau, ta xem chúng là cùng một đoạn. Với hai hàm băm độc lập, xác suất nhầm lẫn rất nhỏ trong bối cảnh bài này.

Do hai đoạn liên tiếp sub_t và sub_{t+1} chỉ khác nhau ở một phần tử bị bỏ ở đầu và một phần tử thêm ở cuối, các giá trị băm có thể cập nhật cuốn chiếu:

$$f_{t+1} = (f_t p_1 - a_t p_1^{\text{len}} + a_{t+\text{len}}) \bmod \text{hashsize}_1,$$

và pos_{t+1} được tính tương tự với p_2 . Vì vậy mọi đoạn độ dài len được xử lý trong $\mathcal{O}(N)$ thời gian. Kết hợp với tìm kiếm nhị phân, độ phức tạp kỳ vọng là $\mathcal{O}(N \log N)$.

Khung thuật toán.

```
check(len):
    clear hash table
    compute rolling hashes for all length-len substrings
    insert each pair (pos, f) and count its occurrences
    return true if some count reaches k

answer:
    binary search len from 0 to N using check(len)
```

Nhận xét. Trong ví dụ này, tư duy “phân” xuất hiện qua việc tách bài toán tối ưu dưới các ràng buộc N, k thành bài toán tồn tại dưới các ràng buộc N, k, len . Việc thêm tham số len làm cấu trúc bài toán rõ hơn, từ đó cho ra một thuật toán ngắn gọn và hiệu quả.

2.2 Ví dụ 2: Tái dựng cây

Phát biểu. Cho số lá l của một cây, với $2 \leq l \leq 200$, và khoảng cách trong cây giữa mọi cặp lá. Mọi cạnh của cây có trọng số 1, mọi khoảng cách không vượt quá 200. Hãy tái dựng một cây phù hợp. Các đỉnh chưa biết có thể được đặt nhãn tùy ý. Nếu không tồn tại cây như vậy, in -1 .

Bài này lấy từ ZOJ 2701, *Restore the Tree*.

Phân tích các trường hợp nhỏ. Điều kiện đầu vào chỉ mô tả quan hệ khoảng cách giữa các lá; các đỉnh trong không được mô tả trực tiếp, nên dựng cây ngay từ đầu rất khó. Ta bắt đầu bằng các trường hợp nhỏ.

Nếu chỉ có hai lá, đáp án hiển nhiên là một đường đi đơn giữa hai lá ấy:

$$1 \text{ --- } 2.$$

Với ba lá, đặt $\text{dist}[i, j]$ là khoảng cách giữa lá i và lá j . Để thuận tiện, lấy lá 1 làm gốc suy nghĩ.

Trước hết, nếu

$$\text{dist}[2, 3] > \text{dist}[1, 2] + \text{dist}[1, 3],$$

thì vi phạm bất đẳng thức tam giác trên cây, nên vô nghiệm. Hai bất đẳng thức tương tự với các cặp còn lại cũng cho vô nghiệm.

Nếu

$$\text{dist}[2, 3] = \text{dist}[1, 2] + \text{dist}[1, 3],$$

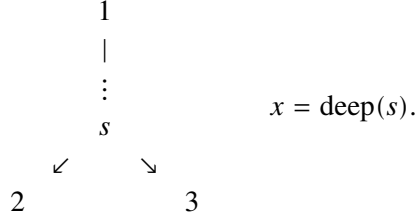
thì lá 1 nằm trên đường đi từ 2 đến 3. Khi đó 1 không thể là lá trong cây cần dựng, mâu thuẫn với đề bài. Các trường hợp bằng nhau tương tự cũng vô nghiệm.

Trường hợp còn lại là các bất đẳng thức đều nghiêm. Khi ấy ba đường đi từ ba lá gặp nhau tại một điểm phân nhánh. Độ sâu của điểm phân nhánh tính từ lá 1 là

$$x = \frac{\text{dist}[1, 2] + \text{dist}[1, 3] - \text{dist}[2, 3]}{2}.$$

Do đó nếu $\text{dist}[1, 2] + \text{dist}[1, 3] - \text{dist}[2, 3]$ là số lẻ, x không nguyên và bài toán vô nghiệm. Nếu chẵn, lá 3 nằm ở cuối một nhánh tách ra từ đỉnh thứ x trên đường đi từ lá 1 đến lá 2, với độ dài nhánh là $\text{dist}[1, 3] - x$.

Có thể hình dung dạng cây ba lá như sau:



Phân tích tiếp. Từ trường hợp ba lá, ta rút ra một nhận xét quan trọng: hai lá bất kỳ luôn được nối bởi một đường đi gồm các đỉnh trong, và vị trí của mọi lá còn lại so với đường đi ấy đều có thể xác định bằng công thức ba lá ở trên.

Gọi bài toán với n lá là $P(n)$. Nếu biết cách ghép một lá mới vào một cây đã dựng, thì $P(n)$ chuyển thành $P(n - 1)$. Cụ thể, giả sử cây hiện tại T đã chứa $k - 1$ lá, mang nhãn $1, 2, \dots, k - 1$. Lấy lá 1 làm gốc, và cần ghép thêm lá k .

Với mỗi lá $i \neq 1$ đã có trong T , xét ba lá $1, i, k$. Công thức trên cho ta điểm phân nhánh s_i của lá k đối với đường đi $1 \rightarrow i$, cùng độ sâu $\text{deep}(s_i)$:

$$\text{deep}(s_i) = \frac{\text{dist}[1, i] + \text{dist}[1, k] - \text{dist}[i, k]}{2}.$$

Nếu cây thật tồn tại, mọi điểm s_i này đều nằm trên cùng đường đi từ 1 đến k . Khi ghép lá mới, phần cây mới duy nhất là đoạn từ điểm phân nhánh sâu nhất đến lá k . Đặt

$$s_{\max} = \arg \max_i \text{deep}(s_i).$$

Ta tách từ $s_{\max}$ một đường mới dài

$$\text{dist}[1, k] - \text{deep}(s_{\max})$$

và đặt cuối đường ấy là lá k .

Quan sát này cũng giúp giảm mạnh gánh nặng xét trường hợp. Trong quá trình ghép, số tình huống vô nghiệm khá nhiều; nếu xử lý hết ngay tại từng bước, tư duy và cài đặt đều phức tạp. Nhưng nếu đề có nghiệm, cách ghép trên sẽ đưa lá mới vào đúng vị trí. Vì vậy ta có thể dựng một cây ứng viên trước, rồi kiểm tra toàn bộ ở cuối.

Khung thuật toán.

1. Dựng cây cơ sở T , ban đầu chỉ gồm lá 1, lá 2, và một đường đi gồm $\text{dist}[1, 2]$ cạnh nối chúng.
2. Lấy một lá chưa có trong T . Với mỗi lá cũ $i \neq 1$, tính điểm phân nhánh s_i theo công thức ba lá. Chọn điểm sâu nhất $s_{\max}$, rồi gắn lá mới bằng một đường có độ dài $\text{dist}[1, k] - \text{deep}(s_{\max})$.
3. Lặp đến khi mọi lá đã được gắn.
4. Kiểm tra cây ứng viên: số lá phải đúng bằng l , và với mọi cặp lá (i, j) , khoảng cách trong cây phải bằng dữ liệu $\text{dist}[i, j]$. Nếu kiểm tra thất bại thì vô nghiệm; ngược lại cây thu được là một nghiệm hợp lệ.

Với $l \leq 200$ và khoảng cách tối đa 200, số đỉnh tạo ra vẫn cùng bậc với tổng độ dài các nhánh. Cách cài đặt thông thường duy trì độ sâu, cha và bằng vị trí trên các đường đi cho độ phức tạp thời gian và bộ nhớ $\mathcal{O}(l^2)$, đạt đến mức cần thiết vì bản thân dữ liệu khoảng cách đã có kích thước $\Theta(l^2)$.

Nhận xét. Ở bài này, ta đi từ trường hợp nhỏ để phát hiện tính chất bản chất, rồi dùng “phân” và “hợp” để chuyển bài toán n lá thành bài toán $n - 1$ lá cộng với một thao tác ghép lá. Chính việc gom lại ở cuối bằng bước kiểm tra toàn cục cho phép bỏ qua nhiều nhánh xét riêng lẻ trong quá trình dựng.

2.3 Ví dụ 3: Dãy tối ưu

Phát biểu. Cho một dãy số nguyên dương độ dài n , với $n \leq 1000$. Cần chọn một dãy con sao cho trong mọi đoạn liên tiếp độ dài m , số phần tử được chọn không vượt quá k , trong đó $k, m \leq 100$. Tổng các phần tử được chọn phải lớn nhất.

Phân tích ban đầu. Thoạt nhìn, quy hoạch động là công cụ tự nhiên cho bài toán tối ưu kiểu này. Nhưng ràng buộc trên mọi đoạn độ dài m làm trạng thái trở nên nặng. Nếu nén trạng thái theo m vị trí gần nhất thì độ phức tạp có thể lên tới $\mathcal{O}(2^m n)$, không thể dùng. Cây đoạn cũng phù hợp với thao tác trên đoạn một chiều, nhưng sau nhiều thử nghiệm vẫn không xử lý được ràng buộc toàn cục của bài toán.

Khó khăn chính nằm ở việc k và m đều khá lớn, khiến quan hệ giữa các lựa chọn quá phức tạp. Ta tách bài toán thành một phiên bản đơn giản hơn.

Tách phức tạp thành đơn giản. Xét bài toán con: chọn một dãy con sao cho trong mọi đoạn độ dài m , có không quá 1 phần tử được chọn, và tổng là lớn nhất.

Bài toán con này có quy hoạch động rất gọn. Đặt $dp[i]$ là tổng lớn nhất có thể lấy trong đoạn $[1, i]$. Khi xét phần tử i , hoặc không lấy nó, hoặc lấy nó và khi đó phần tử được chọn trước đó phải nằm trước $i - m + 1$:

$$\begin{aligned} dp[i] &= \max(dp[i - 1], dp[i - m] + \text{value}[i]) & (i \geq m), \\ dp[i] &= \max(dp[i - 1], \text{value}[i]) & (0 < i < m), \\ dp[0] &= 0. \end{aligned}$$

Câu hỏi là bài toán con này liên hệ thế nào với bài toán gốc.

Một mệnh đề cấu trúc. Một nghiệm của bài toán gốc tương đương với hợp của k nghiệm bài toán con đôi một không giao nhau.

Định lý 1. Mọi nghiệm của bài toán gốc đều có thể được phân hoạch thành k nghiệm con không giao nhau của phiên bản “mỗi đoạn độ dài m chọn không quá 1 phần tử”.

Chứng minh. Lấy một nghiệm bất kỳ

$$P = \{a_1, a_2, \dots, a_t\}, \quad a_1 < a_2 < \dots < a_t.$$

Gọi $\text{sum}[i]$ là số phần tử được chọn trong đoạn $[1, i]$. Theo đề bài,

$$\text{sum}[i] - \text{sum}[i - m] \leq k \quad (i \geq m).$$

Từ đó suy ra với mọi $j > k$,

$$a_j - a_{j-k} \geq m.$$

Bây giờ chia P thành k nhóm theo chỉ số modulo k :

$$P_r = \{a_j : j \equiv r \pmod{k}\}, \quad 0 \leq r < k.$$

Trong mỗi nhóm P_r , hai phần tử liên tiếp cách nhau ít nhất m , nên mỗi đoạn độ dài m chứa không quá một phần tử của nhóm ấy. Vì vậy mỗi P_r là nghiệm hợp lệ của bài toán con, và các nhóm không giao nhau, hợp lại đúng bằng P .

Định lý 2. Hợp của k nghiệm con đôi một không giao nhau luôn là một nghiệm hợp lệ của bài toán gốc.

Chứng minh. Gọi các nghiệm con là $P_0, P_1, \dots, P_{k-1}$. Trong mỗi P_i , hai phần tử được chọn liên tiếp cách nhau ít nhất m , nên một đoạn bất kỳ độ dài m chứa nhiều nhất một phần tử của P_i . Vì thế đoạn ấy chứa nhiều nhất k phần tử của

$$P = P_0 \cup P_1 \cup \dots \cup P_{k-1}.$$

Do đó P thỏa ràng buộc của bài toán gốc.

Hai định lý trên cho thấy điều kiện cần và đủ: nghiệm gốc chính là cách gom k nghiệm con không giao nhau.

Từ bài toán con đến mô hình tổng thể. Ta đã chuyển “một nghiệm tối ưu” thành “hợp tối ưu của k nghiệm con”. Vấn đề còn lại là tìm được k nghiệm con không giao nhau có tổng lớn nhất.

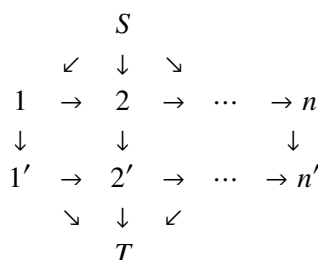
Không thể chỉ chạy quy hoạch động bài toán con k lần, vì các nghiệm con có thể chọn trùng một phần tử. Tham lam theo từng nghiệm con cũng không đúng: k nghiệm tối ưu cục bộ khi hợp lại chưa chắc tạo thành nghiệm toàn cục.

Đây là lúc cần tư duy “hợp”. Thay vì xử lý từng nghiệm con riêng biệt, ta xây dựng một mô hình dòng chi phí cực đại để xét đồng thời mọi nghiệm con. Những bài toán có ràng buộc “mỗi điểm được chọn tối đa một lần” thường phù hợp với mạng dòng.

Mô hình dòng. Dựng mạng có hướng có trọng số $D = (V, A, C)$:

- Mỗi phần tử i của dãy được biểu diễn bằng hai đỉnh i và i' . Thêm cung $i \rightarrow i'$ có dung lượng 1, chi phí $\text{value}[i]$. Đi qua cung này tương ứng với chọn phần tử i .
- Với mọi i , thêm cung $i \rightarrow i+1$ có dung lượng vô hạn và chi phí 0. Cung này cho phép bỏ qua phần tử.
- Nguồn S nối đến mọi đỉnh i bằng cung dung lượng vô hạn, chi phí 0.
- Mọi đỉnh i' nối đến đích T bằng cung dung lượng vô hạn, chi phí 0.
- Với mọi i sao cho $i+m \leq n$, thêm cung $i' \rightarrow i+m$ có dung lượng vô hạn, chi phí 0. Cung này bảo đảm sau khi chọn i , cùng một đơn vị dòng chỉ có thể chọn tiếp từ vị trí cách ít nhất m .

Sơ đồ khái niệm là



trong đó các cung $i \rightarrow i'$ mang lợi ích $\text{value}[i]$, còn cung $i' \rightarrow i+m$ thể hiện khoảng cách tối thiểu giữa hai phần tử do cùng một nghiệm con chọn.

Mỗi đơn vị dòng từ S đến T tương ứng với một nghiệm con của phiên bản “không quá một phần tử trong mọi đoạn độ dài m ”. Vì cung $i \rightarrow i'$ có dung lượng 1, một phần tử không thể được chọn bởi hai nghiệm con khác nhau. Do đó chỉ cần tìm k lần đường tăng chi phí lớn nhất, hoặc tương đương gửi k đơn vị dòng có tổng chi phí lớn nhất, ta thu được đáp án.

Đồ thị có số cung cùng bậc với số đỉnh. Nếu dùng SPFA để tìm đường tăng theo chi phí lớn nhất trong mạng dư, độ phức tạp kỳ vọng khoảng $\mathcal{O}(kN)$, rất tốt với giới hạn của bài.

Khung thuật toán.

build graph:

```
for i = 1..n:
    add edge i -> i' with capacity 1 and cost value[i]
    add edge S -> i with infinite capacity and cost 0
    add edge i' -> T with infinite capacity and cost 0
    if i < n: add edge i -> i+1 with infinite capacity and cost 0
    if i+m <= n: add edge i' -> i+m with infinite capacity and cost 0
```

answer:

```
repeat k times:
    find a maximum-cost augmenting path from S to T
    augment one unit of flow
return total cost
```

Nhận xét. Khi bài toán khó tiếp cận, ta dùng “phân” để đơn giản hóa, qua đó nhận ra cấu trúc của nghiệm. Sau đó dùng “hợp” để đưa mọi bài toán con vào một mô hình dòng duy nhất. Ví dụ này cho thấy vai trò rất mạnh của hai thao tác ấy trong tư duy giải bài.

3 Kết luận

Bài viết đã giới thiệu vài cách dùng tư duy “phân” và “hợp”: khi bài toán có tính đơn điệu, tách theo tham số để dùng tìm kiếm nhị phân; khi phát hiện tính chất quy nạp, bắt đầu từ phạm vi nhỏ rồi mở rộng dần; và khi bài toán khó quan sát trực tiếp, đơn giản hóa để tìm cấu trúc rồi gom các phần lại bằng một mô hình tổng thể.

Trong ví dụ *Mẫu sữa*, còn nhiều lời giải hay khác, nhưng cách tiếp cận từ “phân” có độ khó tư duy thấp, dễ cài đặt và phù hợp với môi trường thi cần tốc độ. Trong ví dụ *Tái dựng cây*, quy về bài toán nhỏ hơn gần như là con đường bắt buộc. Hai ví dụ đầu cũng cho thấy “phân” và “hợp” tuy đối lập nhưng không có ranh giới cứng: đã phân thì phải có lúc hợp lại, và trong hợp vẫn có phân.

Tư duy “phân” giúp nhanh chóng đi vào lõi bài toán, nhưng nếu chia quá vụn thì vấn đề trở nên rời rạc và mất phương hướng. Tư duy “hợp” xâu các phần phức tạp lại thành một chỉnh thể, làm nổi bật quan hệ giữa chúng. Hai mặt ấy bổ sung cho nhau.

Những ví dụ trong bài chỉ chạm đến một phần nhỏ của chủ đề. Chia để trị, chuyển hóa qua phần bù và nhiều cấu trúc dữ liệu cao cấp đều là những ứng dụng quen thuộc của cùng một tinh thần. Quan trọng hơn, “phân” và “hợp” không chỉ là kỹ thuật cài đặt mà còn là phương pháp tư duy. Dù tách hay gom, mục tiêu đều là chuyển bài toán về hình thức dễ suy nghĩ hơn. Kết hợp kiến thức sẵn có với tư duy chuyển hóa, đồng thời tích lũy kinh nghiệm và dám thử cách nhìn mới, ta mới có thể tìm ra bản chất trong các bài toán biến hóa và giải chúng nhanh hơn, hiệu quả hơn.

Lời cảm ơn

Tác giả cảm ơn cô Wei Lizhen đã hướng dẫn và giúp đỡ trong quá trình viết bài; cảm ơn anh Wang Zhikun đã đọc kỹ và góp ý chỉnh sửa; cảm ơn bạn Yuan Xinhao đã cung cấp ví dụ thứ ba; và cảm ơn bạn Wang Hang vì sự giúp đỡ trong quá trình viết.

Tài liệu tham khảo

[A] Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*.

[B] Yu Yuanming, *Thuật toán đường đi ngắn nhất và ứng dụng*, luận văn trại đông 2006.

A Đề bài Mẫu sữa

Farmer John nhận thấy chất lượng sữa của đàn bò thay đổi từng ngày. Sau khi tìm hiểu thêm, ông phát hiện tuy không thể dự đoán chính xác chất lượng sữa của ngày kế tiếp, dữ liệu vẫn có những mẫu lặp đều đặn.

Mỗi mẫu sữa được ghi thành một số nguyên từ 0 đến 1 000 000. Với dữ liệu của một con bò trong N ngày, $1 \leq N \leq 20\,000$, hãy tìm mẫu dài nhất xuất hiện giống hệt ít nhất K lần, với $2 \leq K \leq N$. Các lần xuất hiện có thể chồng lấn; chẳng hạn dãy 1, 2, 3, 2, 3, 2, 3, 1 chứa mẫu 2, 3, 2, 3 hai lần. Dữ liệu bảo đảm có ít nhất một mẫu xuất hiện ít nhất K lần.

Dữ liệu vào. Dòng đầu chứa N và K . N dòng tiếp theo chứa chất lượng sữa từng ngày.

Dữ liệu ra. In một số nguyên: độ dài mẫu dài nhất xuất hiện ít nhất K lần.

Ví dụ.

Input:	8 2
	1
	2
	3
	2
	3
	2
	3
	1
Output:	4

B Đề bài Tái dựng cây

Một cây là đồ thị vô hướng liên thông không có chu trình. Đỉnh có bậc 1 trong cây được gọi là lá. Với một cây, ta có thể xác định khoảng cách giữa mọi cặp lá, tức số cạnh trên đường đi giữa hai lá ấy. Cho tất cả các khoảng cách này, hãy tái dựng cây.

Dữ liệu gồm nhiều bộ. Mỗi bộ bắt đầu bằng l , số lá của cây, với $2 \leq l \leq 200$. Tiếp theo là ma trận $l \times l$ khoảng cách giữa các lá. Mọi khoảng cách không âm, không vượt quá 200, đường chéo chính bằng 0, và ma trận đối xứng.

Nếu không tồn tại cây phù hợp, in -1 . Ngược lại, in số đỉnh n , rồi in $n - 1$ cạnh của cây. l đỉnh đầu tiên phải tương ứng với các lá trong dữ liệu; các đỉnh khác được đánh số tùy ý. Nếu có nhiều nghiệm, in nghiệm bất kỳ.

Ví dụ.

```
Input:  3
        0 2 3
        2 0 3
        3 3 0
Output:  5
        1 4
        2 4
        3 5
        4 5
```